Bayesian Machine Learning in Quantitative Finance

Chapter 6: Credit Ratings via Sparse/Distributed Gaussian Processes

Based on Mongwe, Mbuvha & Marwala

July 13, 2026

Chapter Overview

1. 6.1 Introduction
2. 6.2 Machine Learning Models
3. 6.3 Numerical Experiments
4. 6.4 Results and Discussions
5. 6.5 Conclusion

6.1 Introduction: Why Credit Ratings? I

Intuition

A **credit rating** is a shorthand answer to the question: "how likely is this company to fail to pay back what it owes?" Agencies compress a huge amount of financial detail into a single letter grade (AAA, BBB, CCC, ...) that investors and lenders can act on quickly.

- Credit ratings emerged in 1860 when H.V. Poor began publishing financial data on US railroad firms; J. Moody later grouped this information into alphabetical bands.
- Today, at least one US credit rating agency rates 100% of commercial paper and 99% of corporate bonds.
- Ratings feed directly into investment decisions by bond investors, debt issuers, government officials, and lenders — so they must be as accurate as possible.
- Manual expert assessment is thorough but slow and expensive, which motivates **automated** rating prediction.

6.1 Introduction: Why Credit Ratings? II

Two families of automated approaches in the literature:

- **Traditional statistical models**: logistic regression, linear discriminant analysis, Bayesian networks. These make distributional assumptions and often struggle with non-linear input–output relationships.
- **Machine learning models**: decision trees, neural networks, Naïve Bayes, support vector machines, ensembles. These make fewer distributional assumptions and tend to improve with more data, but few studies bring a **Bayesian** lens to credit ratings.

Why go Bayesian?

A Bayesian approach does not just give a point estimate ("BBB") — it also gives a **measure of uncertainty** around that prediction, which is valuable when the decision (lend or not lend, price a bond) carries real financial risk.

6.1 What This Chapter Does

First-in-literature contribution

A Bayesian analysis of US corporate credit ratings using **sparse** and **distributed** Gaussian processes (GPs).

- **Sparse GPs**: trained with *variational inference* by maximizing a lower bound related to the log marginal likelihood.
- **Distributed GPs**: several *product-of-experts* variants that combine many small, local GP experts through a tractable product rule; the kernel is **shared** across experts.
- An **automatic relevance determination (ARD)** kernel is used throughout, so the reciprocal of each input's kernel length-scale gives a natural **feature-importance ranking** — useful for stakeholders who want to know *what drives* a rating.

Roadmap: §6.2 the models (MLR benchmark, sparse GP, distributed GP) → §6.3 the data and metrics → §6.4 results → §6.5 conclusion.

6.2 Three Models, One Task

The task

Given a company's financial ratios, predict which **risk category** (a small number of ordered classes) its credit rating falls into.

- **6.2.1 Multinomial logistic regression (MLR)** — the classical multi-class *benchmark* model.
- **6.2.2 Sparse Gaussian processes** — a Bayesian non-parametric model made tractable on large data via *inducing points*.
- **6.2.3 Distributed Gaussian processes** — scale GPs differently: split the data, train many small "expert" GPs, and *combine* their predictions.

6.2.1 Multinomial Logistic Regression – Intuition

Plain-language idea

Ordinary (binary) logistic regression gives you *one* probability: the chance of "yes" vs. "no." **Multinomial logistic regression (MLR)** is the multi-class generalization: instead of one probability, you get a probability for *each* possible category (e.g. Low / Medium / High risk), computed via the **softmax** function, and these probabilities always sum to 1.

The book's three defining assumptions of MLR:

1. There is a *linear* relationship between the covariates and the link-function-transformed outcome.
2. Observations are independent of each other.
3. Outcomes follow a **categorical distribution** produced from the covariates via a link function.

The link function used is the **logit** (log-odds), which constrains every predicted probability to lie in $[0, 1]$.

6.2.1 Deriving the Softmax Probability

Setup

Suppose there are K risk categories (classes) $1, \ldots, K$, and each class k has its own weight vector $\mathbf{w}_k \in \mathbb{R}^p$ acting on a feature vector $\mathbf{x} \in \mathbb{R}^p$ (which includes a leading 1 for the intercept). Define the **linear score (logit)** for class k:

$$z_k = \mathbf{w}_k^\top \mathbf{x}$$

Softmax / MLR probability

$$\mathbb{P}(y = k \mid \mathbf{x}) = \frac{\exp(z_k)}{\sum_{\ell=1}^{K} \exp(z_\ell)} = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{\ell=1}^{K} \exp(\mathbf{w}_\ell^\top \mathbf{x})}$$

Symbol guide: z_k is the raw (unnormalized) score for class k; exponentiating makes every score positive; dividing by the sum *normalizes* the scores so they sum to 1 — turning arbitrary real numbers into a valid probability distribution over the K classes. One class (say class 1) is fixed as the **reference class** with $\mathbf{w}_1 = \mathbf{0}$ for identifiability, exactly as ordinary (binary) logistic regression fixes one baseline outcome.

Python: Multinomial Logistic Regression (6.2.1)

```python
import numpy as np

def softmax_probs(x, W):
    """
    x : feature vector, shape (p,), leading 1 for intercept
    W : weight matrix, shape (K, p) -- one row w_k per class
        (row for the reference class is all zeros)
    Returns the softmax probability for each of the K classes.
    """
    logits = W @ x                          # z_k = w_k^T x for every class k
    logits -= logits.max()                  # subtract max: numerical stability
    exp_logits = np.exp(logits)
    return exp_logits / exp_logits.sum()

# Toy example: 3 classes (Low, Medium, High risk), 4-dim feature vector
x = np.array([1.0, 0.40, -0.60, 1.30])        # [bias, debtRatio_z, profitMargin_z, currentRatio_z
    ]
W = np.array([
    [ 0.00,  0.00,  0.00,  0.00],              # reference class: Low risk
    [-0.30,  1.10, -0.50, -0.20],              # Medium risk
    [-0.80,  2.30, -1.40, -0.90],              # High risk
])
probs = softmax_probs(x, W)
print(probs)    # -> [0.332, 0.398, 0.269]  (sums to 1.0)
```

Toy Running Example: 5 Fictional Companies

Setup (illustrative only – not the book's real dataset)

Five made-up companies, each described by three standardized financial ratios (z-scored so mean ≈ 0, unit scale): debt ratio, profit margin, and current ratio.

Company	**debtRatio**$_z$	**profitMargin**$_z$	**currentRatio**$_z$
Acme Corp	0.40	-0.60	1.30
Bolt Industries	1.50	-1.20	-0.80
Cinder Ltd	-0.90	1.10	0.20
Delta Freight	0.10	0.30	-0.10
Echo Systems	-1.10	0.40	-0.60

Toy weights (relative to reference class "Low risk", as in the code above):
$\mathbf{w}_{\text{Medium}} = (-0.30,\ 1.10,\ -0.50,\ -0.20)$, $\mathbf{w}_{\text{High}} = (-0.80,\ 2.30,\ -1.40,\ -0.90)$.

Toy Worked Example: Softmax by Hand for Acme Corp

Feature vector (with leading 1 for the intercept):

$$\mathbf{x}_{\text{Acme}} = (1,\ 0.40,\ -0.60,\ 1.30)$$

Step 1 – compute the three logits $z_k = \mathbf{w}_k^\top \mathbf{x}$:

$$\begin{aligned} z_{\text{Low}} &= 0 \quad \text{(reference class, fixed at 0)} \\ z_{\text{Medium}} &= (-0.30)(1) + (1.10)(0.40) + (-0.50)(-0.60) + (-0.20)(1.30) = 0.18 \\ z_{\text{High}} &= (-0.80)(1) + (2.30)(0.40) + (-1.40)(-0.60) + (-0.90)(1.30) = -0.21 \end{aligned}$$

Step 2 – exponentiate:

$$e^{0} = 1.000, \qquad e^{0.18} = 1.197, \qquad e^{-0.21} = 0.811$$

Step 3 – normalize (divide by the sum $1.000 + 1.197 + 0.811 = 3.008$):

$$\mathbb{P}(\text{Low}) = \frac{1.000}{3.008} = 0.332, \quad \mathbb{P}(\text{Medium}) = \frac{1.197}{3.008} = 0.398, \quad \mathbb{P}(\text{High}) = \frac{0.811}{3.008} = 0.269$$

Prediction: pick the largest probability $\Rightarrow$ **Medium risk** (0.398), even though it is a close call against Low risk (0.332).

Toy Example: Softmax Probabilities & All 5 Companies

Applying the same formula to all 5 toy companies:

Company	$\mathbb{P}$(Low)	$\mathbb{P}$(Med)	$\mathbb{P}$(High)
Acme Corp	0.332	**0.398**	0.269
Bolt Ind.	0.006	0.050	**0.944**
Cinder Ltd	**0.860**	0.131	0.009
Delta Fr.	**0.469**	0.340	0.191
Echo Sys.	**0.807**	0.165	0.028

Bold = predicted class (largest probability). Every row sums to 1.0, exactly as the softmax construction guarantees.

6.2.2 Sparse Gaussian Processes – Intuition

Recap: what is a Gaussian process?

A **Gaussian process (GP)** is a non-parametric, probabilistic model: a prior over *functions*, fully specified by a mean function $\mu(\mathbf{x}')$ and a kernel (covariance) function $k(\mathbf{x}'_i, \mathbf{x}'_j)$. Unlike a deep neural network, a GP naturally produces **uncertainty** around every prediction, not just a point estimate.

The scaling problem

Fitting an exact GP requires **inverting an** $n \times n$ **matrix** built from all n training points — an $O(n^3)$ operation. With thousands of companies and dozens of financial ratios, this quickly becomes infeasible.

Sparse GP idea: instead of using all n training points, summarize the dataset with a much smaller set of $m \ll n$ **inducing points** (pseudo-training examples). The number of inducing points is a user-chosen dial that trades off approximation quality against memory and compute cost.

6.2.2 GP Regression: Mean and Variance (Eqs. 6.1–6.2)

Setup (as in the book)

Training data $\{\mathbf{x}'_t, \mathbf{x}_t\}_{t=1}^{t}$ with noisy observations $\mathbf{x}_t = f(\mathbf{x}'_t) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma_X^2)$. The GP prior on f is $f(\mathbf{x}') \sim \mathcal{N}(\mu_{x'}, \mathbf{K}_{x'} + \sigma_X^2 \mathbf{I})$.

Posterior predictive mean and variance at test point $\mathbf{x}'_*$

$$\mathbb{E}(f_*) = \mu_{\mathbf{x}'_*} + \mathbf{k}_*(K_{x'} + \sigma_X^2 \mathbf{I})^{-1}(\mathbf{X} - \mu_{x'}) \tag{6.1}$$

$$\mathrm{var}(f_*) = k_{**} - \mathbf{k}_*^\top (K_{x'} + \sigma_X^2 \mathbf{I})^{-1} \mathbf{k}_* \tag{6.2}$$

Symbol guide: $k_{**} = k(\mathbf{x}'_*, \mathbf{x}'_*)$ is the prior variance at the test point; $\mathbf{k}_* = k(\mathbf{x}', \mathbf{x}'_*)$ is the vector of covariances between the training points and the test point; θ (kernel hyperparameters) and σ_X (noise) are fit by maximizing the **log-marginal likelihood** $\log p(\mathbf{X} \mid \mathbf{X}') = \log \mathcal{N}\big(\mu_{x'}, \mathbf{K}_{x'x'} + \sigma_X^2 \mathbf{I}\big)$ (Eq. 6.3).

6.2.2 Making It Sparse: Inducing Points & Variational Inference

- **Inducing inputs** (pseudo-training examples) lower the rank of the matrix that must be inverted, cutting the cubic cost.
- Too few inducing points may miss local structure in fast-changing regions of the function; the user controls this trade-off.
- Sparse GPs generally **lack a closed-form solution**, so **variational inference** is used: Bayesian inference is recast as an optimization problem — maximizing a **lower bound** on the log marginal likelihood.
- The locations of the pseudo-training points are treated as optimization arguments of the approximate posterior, learned *jointly* with the model hyperparameters (likelihood and prior). This combination is the **sparse variational GP (SVGP)**.

Why this matters for credit ratings

With thousands of company-year observations across 26 financial and sector features, exact GPs are computationally out of reach — SVGP with e.g. 100 or 500 inducing points keeps the model tractable.

Python: Sparse GP Predictive Mean/Variance (6.2.2)

```python
import numpy as np

def sparse_gp_predict(Z, m_u, S_u, x_star, kernel, jitter=1e-6):
    """
    Toy sparse (variational) GP prediction using m inducing points.
    Z       : inducing input locations, shape (m, d)
    m_u     : variational mean at inducing points, shape (m,)
    S_u     : variational covariance at inducing points, shape (m, m)
    x_star : test input, shape (d,)
    kernel : covariance function k(x1, x2)
    """
    Kzz = np.array([[kernel(z_i, z_j) for z_j in Z] for z_i in Z])
    Kzz += jitter * np.eye(len(Z))
    Kzs = np.array([kernel(z_i, x_star) for z_i in Z])        # (m,)
    Kss = kernel(x_star, x_star)

    Kzz_inv = np.linalg.inv(Kzz)
    A = Kzz_inv @ Kzs                                          # (m,)

    mean = A @ m_u                                             # Eq. 6.1 analogue
    var  = Kss - Kzs @ Kzz_inv @ Kzs + A @ S_u @ A              # Eq. 6.2 analogue
    return mean, max(var, 0.0)

# In practice (e.g. GPflow), inducing locations Z, m_u, S_u, and kernel
# hyperparameters theta are all learned jointly by maximizing the ELBO
# (evidence lower bound on the log marginal likelihood).
```

6.2.3 Distributed Gaussian Processes – Intuition

Plain-language idea

Instead of shrinking the data (as sparse GPs do), **distributed GPs** split a huge dataset into chunks, train a separate, smaller GP "expert" on each chunk, and then **combine** their predictions — like asking several specialists for their opinion and pooling the answers. This lets the whole system scale to big data, and training can be spread across different compute clusters.

- Each of M experts is trained on its own data subset $D^j = \{\mathbf{x}^j, y^j\}$.
- Crucially, all experts **share the same kernel hyperparameters** θ — this automatically regularizes the population, since no single expert can overfit its local subset.
- Assuming conditional independence across experts given the data, the shared log marginal likelihood is (Eq. 6.4):

$$\log p(y \mid X, \theta) = \sum_{j=1}^{J} \log p_j(y \mid x^j, \theta^j)$$

- This chapter compares four combination rules: **PoE**, **gPoE**, **BCM**, **rBCM** – all differ only in *how* the experts' Gaussian predictions are recombined.

6.2.3 Generalized Product of Experts (gPoE)

Combining M expert predictions at test point x_* (Eq. 6.5)

$$p(f_* \mid x_*) \propto \prod_{j=1}^{J} p_j^{\beta_j(x_*)}(f_* \mid x_*, D^j)$$

Predictive mean and precision (Eqs. 6.6–6.7)

$$m_{(g)poe}(x_*) = \sigma^2_{(g)poe}(x_*) \sum_{j=1}^{J} \beta_j(x_*)\, \sigma_j^{-2}(x_*)\, m_j(x_*)$$

$$\sigma^{-2}_{(g)poe}(x_*) = \sum_{j=1}^{J} \beta_j(x_*)\, \sigma_j^{-2}(x_*)$$

Symbol guide: $D^j = \{x^j, y^j\}$ is the data given to expert j; $\beta_j(x_*)$ weights expert j's contribution at x_* (typically reflecting how confident that expert is there); setting $\beta_j(x_*) = 1$ for all j recovers the plain **Product of Experts (PoE)**. As the number of experts grows, the product's aggregated variance can vanish, giving **overconfident** predictions – a key weakness of PoE/gPoE.

6.2.3 Robust Bayesian Committee Machine (rBCM)

Predictive distribution via repeated Bayes' rule (Eq. 6.8)

Assuming conditional independence $D_i \perp\!\!\!\perp D_j \mid f_*$:

$$p(f_* \mid x_*) = \frac{\prod_{j=1}^{J} p_j^{\beta_j(x_*)}(f_* \mid x_*, D^j)}{p_j^{-1+\sum_{j=1}^{J}\beta_j(x_*)}(f_* \mid x_*)}$$

Predictive mean and precision (Eqs. 6.9–6.10)

$$m_{(r)bcm}(x_*) = \sigma^2_{(r)bcm}(x_*) \sum_{j=1}^{J} \beta_j(x_*)\, \sigma_j^{-2}(x_*)\, m_j(x_*)$$

$$\sigma^{-2}_{(r)bcm}(x_*) = \sum_{j=1}^{J} \beta_j(x_*) \big(\sigma_j^{-2}(x_*) + \sigma_*^{-2}\big) + \sigma_*^{-2}$$

Setting $\beta_j(x_*) = 1$ for all j recovers the plain **Bayesian Committee Machine (BCM)**. The extra prior-precision term σ_*^{-2} is what guarantees the rBCM/BCM predictive distribution **reverts to the GP prior** away from the training data, unlike PoE/gPoE. This chapter considers all four: PoE, gPoE, BCM, rBCM.

Toy Worked Example: Combining Two Gaussian Experts

Setup: two toy expert predictions at the same test point

Expert 1: $\mathcal{N}(m_1, \sigma_1^2) = \mathcal{N}(2.0,\, 1.00)$ (precision $\sigma_1^{-2} = 1.00$)
Expert 2: $\mathcal{N}(m_2, \sigma_2^2) = \mathcal{N}(3.0,\, 0.25)$ (precision $\sigma_2^{-2} = 4.00$)

Step 1 – combine precisions (PoE special case, $\beta_j = 1$ for all j):

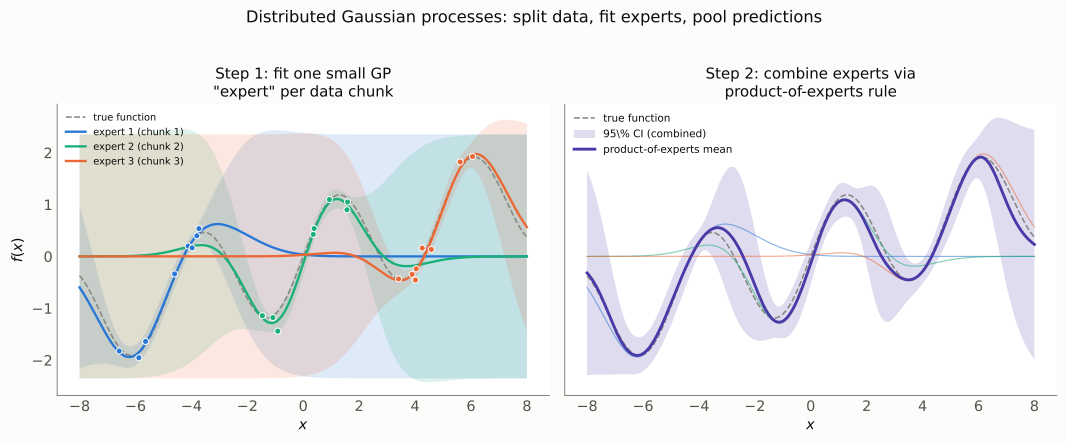
$$\sigma_{poe}^{-2} = \sigma_1^{-2} + \sigma_2^{-2} = 1.00 + 4.00 = 5.00 \implies \sigma_{poe}^2 = \frac{1}{5.00} = 0.20$$

Step 2 – combine means, precision-weighted:

$$m_{poe} = \sigma_{poe}^2 \left(\sigma_1^{-2} m_1 + \sigma_2^{-2} m_2\right) = 0.20 \times \left(1.00 \times 2.0 + 4.00 \times 3.0\right) = 0.20 \times 14.0 = 2.80$$

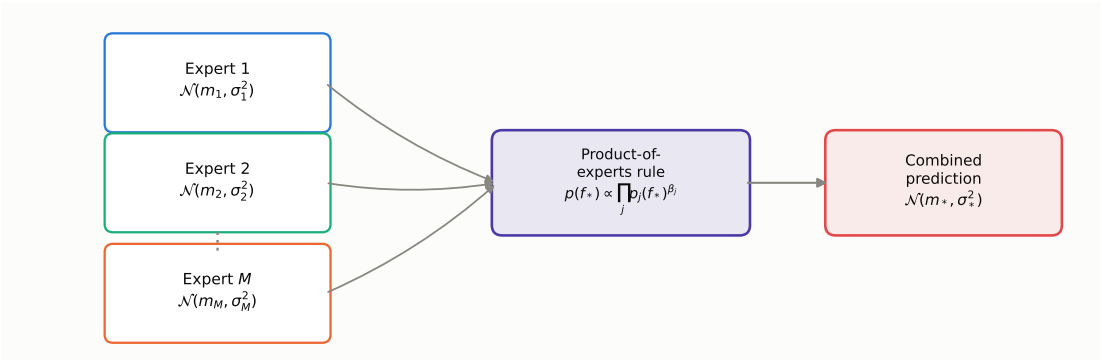
Result: the combined (product-of-experts) prediction is $\mathcal{N}(2.80,\, 0.20)$ — notice the combined variance (0.20) is *smaller* than either expert's variance, and the combined mean (2.80) sits closer to Expert 2's mean because Expert 2 is more confident (smaller variance $\Rightarrow$ higher precision $\Rightarrow$ more "weight" in the average).

Toy Illustration: Distributed GP on 1D Data



A toy dataset is split into 3 chunks; a small GP expert is fitted to each chunk (left); the experts' Gaussian predictions are then combined via the product-of-experts rule into a single aggregated prediction (right, violet). This is the regression analogue of what happens (with a categorical likelihood layered on top) for the credit-rating *classification* task in this chapter.

Toy Illustration: From M Experts to One Prediction



Schematic flow: M local GP experts, each producing its own Gaussian predictive distribution, are pooled via the product-of-experts rule into one combined Gaussian – the mechanism underlying PoE, gPoE, BCM, and rBCM (they differ only in the weights $\beta_j(x_*)$ and the extra prior-precision term).

Python: Product-of-Experts Combination (6.2.3)

```python
import numpy as np

def combine_experts(means, variances, betas=None):
    """
    Generalized product-of-experts combination (Eqs. 6.6-6.7).
    means, variances : arrays of shape (M,) -- one (m_j, sigma_j^2) per expert
    betas : per-expert confidence weights; betas=1 for all j -> plain PoE
    """
    M = len(means)
    if betas is None:
        betas = np.ones(M)                      # plain PoE (Eq. 6.7 with beta_j=1)

    precisions = 1.0 / np.asarray(variances)
    combined_precision = np.sum(betas * precisions)          # Eq. 6.7
    combined_variance  = 1.0 / combined_precision
    combined_mean = combined_variance * np.sum(
        betas * precisions * np.asarray(means))              # Eq. 6.6

    return combined_mean, combined_variance

# Toy two-expert example from the slides
m, v = combine_experts(means=[2.0, 3.0], variances=[1.00, 0.25])
print(m, v)    # -> 2.80  0.20
```

6.3.1 Data Description I

The real dataset used in the book (not the toy example)

2021 credit rating observations on US corporations from the **three major international rating agencies**, each described by **26 features**: 25 financial indicators plus the company's sector.

The 25 financial indicators fall into five groups:

- **Liquidity:** currentRatio, quickRatio, cashRatio, daysOfSalesOutstanding
- **Profitability:** grossProfitMargin, operatingProfitMargin, pretaxProfitMargin, netProfitMargin, effectiveTaxRate, returnOnAssets, returnOnEquity, returnOnCapitalEmployed
- **Debt:** debtRatio, debtEquityRatio
- **Operating performance:** assetTurnover
- **Cash flow:** operatingCashFlowPerShare, freeCashFlowPerShare, cashPerShare, operatingCashFlowSalesRatio, freeCashFlowOperatingCashFlowRatio

6.3.1 Data Description II

Target variable: mapping agency ratings to risk categories

Credit rating(s)	**Risk category**
AAA	Lowest Risk (excluded – too few observations)
AA, A	Low Risk
BBB	Medium Risk
BB, B	High Risk
CCC, CC, C	Highest Risk
D	In Default (excluded)

Entities in default (D) and the very-low-risk AAA band are excluded from the analysis, as both groups had too few observations.

6.3.1 Class Distribution & Pre-processing

Distribution of risk categories (as reported in the book):

- High Risk: 39%
- Medium Risk: 33%
- Low Risk: 24%
- Highest Risk: 4%

The classes are noticeably **imbalanced**, with the "Highest Risk" bucket being rare.

Pre-processing:

- 70/30 train–test split
- Features normalized with a **min–max scaler**
- All models share the same ARD squared-exponential kernel structure (where a kernel is used)

6.3.2 Performance Metrics & Experimental Settings

Metrics on held-out (unseen) test data

Accuracy, **recall**, **precision**, and **F1-score** – higher is better for all four.

Models compared:

- **MLR** – the multinomial logistic regression benchmark (§6.2.1).
- **SVGP** – sparse variational GP, with **100** or **500** inducing points.
- **Distributed GPs** – PoE, gPoE, BCM, rBCM, each with **100** or **200** experts (all using 100 inducing points per expert).

All GP variants use a **squared exponential kernel** with **automatic relevance determination (ARD)**, which lets the chapter extract a feature-importance ranking as a byproduct of training.

6.4 Results: 100 Inducing Points / 100 Experts (Table 6.3)

Metric	MLR	SVGP_{100}	PoE_{100}	gPoE_{100}	BCM_{100}	rBCM_{100}
Accuracy	0.384	0.542	**0.611**	0.606	0.601	0.606
Recall	0.384	0.542	**0.611**	0.606	0.601	0.606
Precision	0.378	0.534	**0.601**	0.596	0.591	0.596
F1	0.309	0.538	**0.605**	0.600	0.594	0.600

Key finding: PoE outperforms across every metric in this setting; **MLR** is the weakest model; every distributed GP variant beats the SVGP model, which in turn beats MLR.

6.4 Results: 500 Inducing Points / 200 Experts (Table 6.4)

Metric	MLR	SVGP_{500}	PoE_{200}	gPoE_{200}	BCM_{200}	rBCM_{200}
Accuracy	0.384	**0.571**	0.532	0.532	0.532	0.532
Recall	0.384	**0.571**	0.532	0.532	0.532	0.532
Precision	0.378	**0.563**	0.516	0.516	0.516	0.516
F1	0.309	**0.566**	0.517	0.517	0.517	0.517

Key finding: increasing the number of **inducing points** (100→500) **improves** SVGP performance as expected – but at a substantial increase in execution time. Increasing the number of **experts** (100→200) instead **degrades** distributed GP performance. Choosing the right number of experts remains an open research problem.

6.4 Which Features Matter Most? (Table 6.5, Figs. 6.2–6.3)

Feature importance via ARD kernel length-scales

For every model, the **reciprocal of each feature's kernel length-scale** gives a natural importance score: the smaller the length-scale, the more sensitive predictions are to that feature, and the more important it is judged to be.

Rank	SVGP_{100}	SVGP_{500}	Distr.GP_{100}	Distr.GP_{200}
1	operatingCashFlowSalesRatio	returnOnAssets	ebitPerRevenue	returnOnCapitalEmployed
2	returnOnCapitalEmployed	operatingCashFlowSalesRatio	operatingCashFlowSalesRatio	operatingCashFlowSalesRatio
3	debtRatio	returnOnCapitalEmployed	returnOnCapitalEmployed	netProfitMargin
4	grossProfitMargin	grossProfitMargin	pretaxProfitMargin	debtRatio
5	returnOnAssets	debtRatio	debtRatio	ebitPerRevenue

Consensus: all models broadly agree; **debtRatio**, **operatingCashFlowSalesRatio**, and profit-margin-related metrics are consistently in the top 5.

6.4 Discussion: Why These Features?

- Firms with **high operatingCashFlowSalesRatio** and strong **profit margins** generate more cash relative to sales, so they are less likely to default – intuitively, this pushes their rating *up* (lower risk).
- Firms with **high debt ratios** are more heavily indebted, which impairs their ability to repay obligations – this pushes their rating *down* (higher risk).
- These results make intuitive financial sense and could be useful to stakeholders (lenders, regulators, investors) who want to understand *what drives* a company's rating, not just the rating itself.

Overall summary of §6.4

Distributed GPs $>$ sparse GP (SVGP) $>$ MLR benchmark, across almost every metric and setting – but distributed GP performance **degrades** as the number of experts grows, an open modeling trade-off.

6.5 Conclusion

Summary

This chapter presented a first-in-literature Bayesian analysis of US corporate credit ratings using **sparse** and **distributed** Gaussian processes, benchmarked against **multinomial logistic regression**.

- Distributed GPs (PoE, gPoE, BCM, rBCM) outperform both the sparse GP and the MLR benchmark across all performance metrics.
- Performance of distributed GPs **degenerates** as more experts are used – the right number of experts is data- and task-dependent.
- The ARD kernel yields feature importances as a useful byproduct: **debt ratio**, **profit-margin** metrics, and **return-on-capital** metrics matter most for US corporate ratings.

Future extensions noted by the authors: try a *mixture* of experts instead of a product of experts; apply the framework to data from a developing nation (e.g. South Africa); study principled ways to choose the number of experts.

Looking ahead: Chapter 7 applies Hamiltonian Monte Carlo to model *recovery rates* on charged-off loans – another setting where understanding a portfolio's risk profile matters for lending decisions.

Key Takeaways I

Multinomial logistic regression (§6.2.1)

- Multi-class generalization of logistic regression via the **softmax** link function.
- Each class gets a probability; all class probabilities sum to 1.
- Serves as the (weaker) benchmark model in this chapter.

Sparse Gaussian processes (§6.2.2)

- Exact GPs cost $O(n^3)$ – infeasible at scale.
- Sparse GPs summarize the data with $m \ll n$ inducing points.
- Trained via variational inference (maximize a lower bound on the log marginal likelihood).

Key Takeaways II

Distributed Gaussian processes (§6.2.3)

- Split data across M experts, sharing kernel hyperparameters.
- Combine expert predictions via PoE, gPoE, BCM, or rBCM.
- PoE/gPoE can be overconfident as M grows; BCM/rBCM correct this by reverting to the prior away from the data.

Key Takeaways III

Data and experiments (§6.3)

- 2021 credit rating observations, US corporations, 26 features (25 financial ratios + sector), mapped to 4 risk categories.
- 70/30 split, min–max scaled, evaluated via accuracy, recall, precision, F1.

Results (§6.4)

- Distributed GPs $>$ SVGP $>$ MLR across almost all settings.
- More inducing points help SVGP; more experts hurt distributed GPs.
- Debt ratio, cash-flow, and profit-margin ratios are the most important predictors across all models.

Further Reading

Key references (this chapter):

- Cohen, Mbuvha, Marwala & Deisenroth (2020). *Healing products of Gaussian process experts*. ICML.
- Leibfried, Dutordoir, John & Durrande (2020). *A tutorial on sparse Gaussian processes and variational inference*. arXiv:2012.13962.
- Rasmussen & Williams (2006). *Gaussian Processes for Machine Learning*. MIT Press.
- Wallis, Kumar & Gepp (2019). *Credit rating forecasting using machine learning techniques*.

Next chapter:
Chapter 7 – Hamiltonian Monte Carlo methods for modeling recovery rates on charged-off loans.

- Portfolio-level lending decisions
- Advanced HMC sampling
- Continuing the Bayesian ML theme in credit risk